

Step 1 - Menu & Category CRUD

Assigned to: **Team Member 1** | Status: Todo list

Goal

Build RESTful CRUD endpoints for categories and menu items. Admin users manage the menu; customers can read it. Follow the existing layered pattern: schemas -> repository -> service -> router.

Files to create / modify

- **app/schemas/menu.py** - CategoryCreate/Update/Out, MenuItemCreate/Update/Out (validation: price ≥ 0 , stock ≥ 0 , min_length on names).
- **app/repositories/menu_repository.py** - CRUD helpers for Category and MenuItem.
- **app/services/menu_service.py** - business logic (409 on duplicate category name, 404 if not found).
- **app/api/v1/menu.py** - router with prefixes /categories and /menu-items.
- **app/api/v1/__init__.py** - register your new router.

Endpoints to implement

- GET /categories - public, returns all categories with their menu items.
- POST /categories - admin only (depend on get_current_admin).
- GET /categories/{id} - public, 404 if missing.
- PATCH /categories/{id} - admin only.
- DELETE /categories/{id} - admin only, cascades to menu items.
- GET /menu-items - public, optional filters: ?category_id= & ?is_available=true.
- POST /menu-items - admin only, validates category exists.
- GET /menu-items/{id} - public, 404 if missing.
- PATCH /menu-items/{id} - admin only.
- DELETE /menu-items/{id} - admin only.

Hints

- Reuse app.api.deps.get_current_user / get_current_admin.
- Keep image_url nullable.
- Use response_model=CategoryOut so relations never leak.
- Models already exist in the DB - import from app.models.

Verification

Create a category, then a menu item under it, update its price, list with the category filter, delete it, and confirm the same flow returns 404 afterwards. Register an admin user in the DB (role='admin') or temporarily promote one to test the admin endpoints. Run the full test suite:

```
backend\venv\Scripts\python.exe -m pytest tests -v
```

Shared Rules

Code conventions

- Layered architecture: schemas -> repositories -> services -> routers. Keep business logic out of routers.
- No comments in code unless truly necessary; follow the style of existing files.
- Routers use APIRouter with explicit tags and response_model.
- Never expose hashed_password or internal fields in responses.
- Auth dependencies: app.api.deps.get_current_user / get_current_admin.

Error handling

- 404 for missing resources, 403 for unauthorized roles, 409 for conflicts.
- Raise HTTPException in services with clear detail messages.
- Validate with pydantic schemas (min_length, ge=0 for money/quantities).

Testing & delivery

- Add tests under tests/ (see tests/test_auth.py as template, incl. cleanup of test rows).
- All tests must pass: backend\venv\Scripts\python.exe -m pytest tests -v
- Commit and push your branch, then report the endpoints you implemented.

Generated for the Smart Canteen team - Phase 2 workstreams.